

# Dynamic Object Motion Analysis (DOMA)

## Rapport de projet : classification de gestes à partir de pose et flot optique

Mourad Latoundji, Le Hoang Nguyen, Chengbo Zhang

10 mars 2026

### Résumé

Ce rapport décrit le pipeline *Dynamic Object Motion Analysis* (DOMA) pour la reconnaissance de gestes de la main à partir de vidéos. Le système combine détection de la main (MediaPipe / YOLO), extraction de flot optique (Farnebäck / RAFT), et plusieurs modèles de classification séquence→classe entraînés sur le jeu IPN Hand. Nous décrivons les méthodes (CNN-LSTM, Temporal Transformer, ST-GCN, ST-GCN-Opt, Temporal ViT), les formules de représentation des entrées (pose, métriques de flot), et nous comparons les performances sur l'ensemble de validation (exactitude, F1 macro/pondéré, perte). Les meilleurs résultats sont obtenus avec CNN-LSTM et Temporal Transformer sur les features unifiées pose+optflow ; le Temporal ViT, entraîné sur des images de flot (entrée différente), atteint environ 73 % d'exactitude.

### Table des matières

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                             | <b>2</b> |
| <b>2</b> | <b>Jeu de données IPN Hand</b>                                  | <b>2</b> |
| <b>3</b> | <b>Pipeline d'extraction des features</b>                       | <b>2</b> |
| 3.1      | Flot optique et métriques . . . . .                             | 2        |
| 3.2      | Représentation pose . . . . .                                   | 3        |
| <b>4</b> | <b>Méthodes (modèles)</b>                                       | <b>3</b> |
| 4.1      | CNN-LSTM . . . . .                                              | 3        |
| 4.2      | Temporal Transformer . . . . .                                  | 4        |
| 4.3      | ST-GCN (Spatial-Temporal Graph Convolutional Network) . . . . . | 4        |
| 4.4      | ST-GCN-Opt (multi-branches) . . . . .                           | 4        |
| 4.5      | Temporal ViT (Vision Transformer temporel) . . . . .            | 5        |
| <b>5</b> | <b>Expériences et résultats</b>                                 | <b>5</b> |
| 5.1      | Protocole . . . . .                                             | 5        |
| 5.2      | Comparaison des modèles . . . . .                               | 5        |
| 5.3      | Analyse . . . . .                                               | 6        |
| <b>6</b> | <b>Conclusion</b>                                               | <b>7</b> |

# 1 Introduction

La reconnaissance continue de gestes de la main (HGR) à partir de vidéos est utile pour les interfaces sans contact, la traduction de signes et le contrôle par gestes. Le projet DOMA vise à :

- restreindre l’analyse du mouvement à une région d’intérêt (ROI) définie par la détection de main (ROI manuelle, MediaPipe Hands ou YOLO) ;
- calculer le flot optique dense dans cette ROI et en extraire des *métriques* (vitesse, direction dominante, concentration) ;
- combiner ces métriques avec des descripteurs de *pose* (position, vitesse, accélération du track 3D, optionnellement 21 landmarks) en un vecteur de features par frame ;
- entraîner des classificateurs séquence→classe sur le benchmark IPN Hand et comparer plusieurs architectures.

Le rapport est structuré comme suit : Section 2 présente le jeu de données IPN Hand ; Section 3 décrit l’extraction des features (pose et flot optique) et les formules associées ; Section 4 détaille les cinq modèles (CNN-LSTM, Temporal Transformer, ST-GCN, ST-GCN-Opt, Temporal ViT) ; Section 5 donne le protocole expérimental et les tableaux de performances ; Section 6 conclut.

## 2 Jeu de données IPN Hand

Le dataset **IPN Hand** [?] est un benchmark vidéo pour la reconnaissance continue de gestes de la main : plus de 4 000 séquences de gestes, 800 000 frames, 50 sujets, résolution 640×480 à 30 fps. Il comporte **14 classes** : une classe *non-geste* (D0X) et 13 gestes (B0A, B0B, G01–G11) conçus pour l’interaction avec des écrans sans contact (pointing, click, throw, zoom, etc.).

Chaque sample est un segment vidéo annoté avec un label. Le pipeline DOMA produit, pour chaque sample, deux fichiers NPZ à partir des vidéos et annotations :

- **Pose / cinématique** (`pose_*.npz`) : position, vitesse et accélération du track 3D (`track_pos_xyz`, `track_vel_xyz`, `track_acc_xyz`), et optionnellement les 21 landmarks main (`landmarks_xyz`) fournis par MediaPipe.
- **Flot optique** (`optflow_*.npz`) : métriques temporelles par frame (vitesse moyenne, vitesse max, angle dominant, concentration de direction, nombre de pixels du masque, seuil).

Un fichier **manifest** (`manifest.csv`) regroupe tous les samples avec les colonnes `sample_id`, `split`, `label`, `pose_npz`, `optflow_npz`. Les modèles sont entraînés en utilisant soit un vecteur unifié pose+optflow par pas de temps, soit des représentations dérivées (skeleton+motion, track) pour les architectures à graphe ou multi-branches.

## 3 Pipeline d’extraction des features

### 3.1 Flot optique et métriques

Le flot optique dense fournit, pour chaque pixel, un vecteur de déplacement  $\mathbf{f} = (u, v)$ . À partir du champ de flot dans la ROI main, on calcule la *magnitude* et l’*angle* :

$$m = \sqrt{u^2 + v^2}, \quad \theta = \text{atan2}(v, u) \in [-\pi, \pi]. \quad (1)$$

Un masque de mouvement est obtenu par seuillage de  $m$  (seuil fixe, Otsu ou MAD). Après filtrage morphologique et sélection de la plus grande composante connexe, on soustrait éventuellement le mouvement global (médiane de  $(u, v)$  en arrière-plan) pour limiter l’effet de la caméra.

- Sur les pixels du masque, on définit les métriques suivantes, stockées dans `optflow_*.npz` :
- **Vitesse moyenne** et **vitesse max** :  $\bar{m}$ ,  $\max m$ .

— **Direction dominante** (moyenne circulaire pondérée par la magnitude) :

$$C = \sum_i w_i \cos \theta_i, \quad S = \sum_i w_i \sin \theta_i, \quad \theta^* = \text{atan2}(S, C), \quad R = \frac{\sqrt{C^2 + S^2}}{\sum_i w_i}. \quad (2)$$

$R \in [0, 1]$  est la *concentration* de direction ;  $\theta^*$  est converti en degrés pour le stockage. On stocke aussi le **nombre de pixels** du masque (optionnel en entrée des modèles) et le **seuil** utilisé pour le masque (Otsu, MAD ou fixe).

### 3.2 Représentation pose

À chaque frame  $t$ , la pose est représentée par :

- **Track 3D** : position  $\mathbf{p}_t$ , vitesse  $\mathbf{v}_t$ , accélération  $\mathbf{a}_t$  (9D par frame). Utilisé dans le vecteur unifié (CNN-LSTM, Temporal Transformer) et comme branche dédiée dans ST-GCN-Opt.
- **Landmarks et skeleton + motion** : les 21 landmarks  $\mathbf{L}_t \in \mathbb{R}^{21 \times 3}$  sont, lorsqu'ils sont disponibles, inclus dans le vecteur unifié par frame (CNN-LSTM, Temporal Transformer), d'où  $F = 78$ . Ils servent aussi à construire la représentation skeleton+motion pour ST-GCN et la branche ST-GCN de ST-GCN-Opt (tenseur  $3 \times T \times 21$ , motion  $\mathbf{M}_t = \mathbf{L}_t - \mathbf{L}_{t-1}$  ; en l'absence de landmarks, un seul nœud à partir du track ( $n = 1$ )).

Les entrées des modèles sont donc toutes dérivées des deux sources **pose** et **optflow** ; selon l'architecture, on utilise soit un vecteur concaténé par frame (Temporal Transformer, CNN-LSTM), soit skeleton+motion et/ou track+optflow (ST-GCN, ST-GCN-Opt), soit des images de flot par frame (Temporal ViT, pipeline ipn\_flow).

## 4 Méthodes (modèles)

On note  $\mathcal{X} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$  la séquence d'entrée (chaque  $\mathbf{x}_t$  est un vecteur de features ou un tenseur selon le modèle),  $\ell$  les longueurs réelles par sample, et  $K$  le nombre de classes. L'objectif est d'estimer une distribution sur les classes  $\hat{y} = \text{argmax}_k \mathbf{z}_k$  à partir des logits  $\mathbf{z} \in \mathbb{R}^K$ . Tous les modèles sont entraînés par minimisation de la perte de cross-entropy :

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \log \frac{\exp(z_{n,y_n})}{\sum_{k=1}^K \exp(z_{n,k})}. \quad (3)$$

### 4.1 CNN-LSTM

**Idée.** On traite la séquence comme une « phrase » de vecteurs (une frame = un mot). La convolution 1D extrait d'abord des motifs locaux dans le temps (petites fenêtres de frames), puis la LSTM lit la séquence dans les deux sens et mémorise les dépendances à longue portée. Une fois toute la séquence parcourue, on résume en une seule représentation (moyenne sur le temps) et on prédit la classe. C'est un classique pour les séries temporelles et les vidéos.

L'entrée est une séquence de vecteurs  $\mathbf{x}_t \in \mathbb{R}^F$  (pose + optflow unifiés,  $F = 78$  par défaut). Le modèle comporte :

1. **Conv1D temporelle** : couches de convolution 1D le long du temps (noyau  $k$ , canaux  $C$ ), BatchNorm, GELU, Dropout, produisant  $\mathbf{H}^{(1)} \in \mathbb{R}^{B \times T \times C}$ .
2. **LSTM bidirectionnelle** :  $\vec{\mathbf{h}}_t, \overleftarrow{\mathbf{h}}_t$  ; sortie concaténée par pas de temps.
3. **Pooling temporel masqué** : moyenne sur les pas valides (non padding) en fonction des longueurs  $\ell$  :

$$\mathbf{h} = \frac{\sum_{t=1}^T \mathbf{m}_t \cdot \mathbf{h}_t}{\sum_t m_t}, \quad m_t = \mathbb{I}[t \leq \ell]. \quad (4)$$

4. **Couche de sortie** : LayerNorm, Dropout, Linear( $C_{\text{lstm}}, K$ )  $\rightarrow$  logits  $\mathbf{z}$ .

## 4.2 Temporal Transformer

**Idée.** Au lieu de parcourir la séquence pas à pas comme la LSTM, on donne au modèle toutes les frames en une fois. Chaque position reçoit un indice de temps (encodage positionnel), puis des couches d'attention permettent à chaque frame de « regarder » toutes les autres pour composer une représentation contextuelle. On agrège ensuite ces représentations (moyenne) et on classe. Très flexible pour les relations à longue distance, mais plus gourmand en données.

L'entrée est la même séquence  $\mathbf{x}_t \in \mathbb{R}^F$ . Le modèle :

1. **Normalisation et projection** :  $\text{LayerNorm}(F)$ ,  $\text{Linear}(F \rightarrow d)$ ,  $\text{LayerNorm}(d)$ , avec  $d$  la dimension du modèle.
2. **Encodage positionnel sinusoïdal** :

$$\text{PE}(t, 2i) = \sin(t/10000^{2i/d}), \quad \text{PE}(t, 2i+1) = \cos(t/10000^{2i/d}). \quad (5)$$

3. **Transformer Encoder** : couches d'attention multi-têtes et FFN (GELU, norm-first), avec masque de padding pour ignorer les pas au-delà de  $\ell$ .
4. **Pooling temporel masqué** : moyenne des sorties du Transformer sur les positions valides, puis  $\text{LayerNorm}$  et classifieur linéaire  $d \rightarrow K$ .

## 4.3 ST-GCN (Spatial-Temporal Graph Convolutional Network)

**Idée.** On représente la main comme un *graphe* : les nœuds sont les articulations (ou le centre de la main), les arêtes relient les parties voisines. À chaque instant, chaque nœud a des features (position, mouvement). Le modèle alterne des convolutions sur le temps (noyau glissant le long des frames) et sur l'espace (propagation le long des arêtes du graphe), ce qui capture à la fois l'évolution temporelle et les relations entre articulations. Adapté aux squelettes et à la reconnaissance d'actions basée pose.

L'entrée est un tenseur  $\mathbf{X} \in \mathbb{R}^{B \times 6 \times T \times n}$  (skeleton et motion concaténés : 3 canaux skeleton + 3 canaux motion,  $T$  pas de temps,  $n$  nœuds,  $n = 21$  avec landmarks). Le graphe spatial est défini par une matrice d'adjacence  $\mathbf{A} \in \mathbb{R}^{n \times n}$  (ici  $\mathbf{A} = \mathbf{I}$  pour un graphe sans arêtes explicites, ou une matrice de voisinage apprise).

### Bloc spatio-temporel (ST-Block).

1. **Convolution temporelle** (noyau  $k_t$ ) avec activation GLU :  $\mathbf{X}' = (\mathbf{W}_t * \mathbf{X}_{:, :, 1:c_1, :} + \mathbf{b}) \odot \sigma(\mathbf{W}'_t * \mathbf{X})$  (partie gate).
2. **Convolution spatiale (graph)** : pour un canal  $c$ ,  $\mathbf{Y}_{b,c,t,n} = \sum_m \mathbf{A}_{n,m} \mathbf{X}_{b,c,t,m}$  puis combinaison linéaire avec noyau spatial  $\Theta$  et biais, puis ReLU et connexion résiduelle.
3. **Deuxième convolution temporelle** (ReLU),  $\text{LayerNorm}$  sur les dimensions  $(n, c)$ , Dropout.

Deux ST-Blocks sont empilés, suivis d'un **pooling global** (moyenne sur temps et nœuds) et d'un classifieur linéaire vers  $K$  classes.

## 4.4 ST-GCN-Opt (multi-branches)

**Idée.** On combine trois « experts » : un qui voit le squelette + mouvement (ST-GCN), un qui voit la trajectoire globale de la main (track 3D), un qui voit les métriques de flot optique. Chaque branche produit un vecteur ; on les fusionne (concaténation, pondération ou attention) puis on classe. L'objectif est de profiter à la fois de la structure articulaire et des signaux globaux (trajectoire, flot), pour des gestes où la forme du squelette et le mouvement d'ensemble comptent.

Ce modèle fusionne trois branches à partir des mêmes données pose + optflow :

1. **Branche ST-GCN** : comme ci-dessus, sur skeleton+motion  $\rightarrow$  vecteur  $\mathbf{h}_{\text{stgcn}}$ .
2. **Branche track** : séquence track (pos, vel, acc) de dimension 9 par frame ; CNN 1D temporelle (Conv1D + BN + ReLU + Dropout) + pooling global  $\rightarrow \mathbf{h}_{\text{track}}$ .
3. **Branche optflow** : séquence des 6 métriques de flot par frame ; même structure CNN 1D  $\rightarrow \mathbf{h}_{\text{opt}}$ .

La fusion peut être *concaténation*, *pondérée* (poids appris + softmax), *attention* (projection commune + multi-head attention) ou *gated*. Les vecteurs fusionnés sont passés dans un MLP (Linear $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout, répété) puis Linear  $\rightarrow$  logits  $K$ .

## 4.5 Temporal ViT (Vision Transformer temporel)

**Idée.** Ici l'entrée n'est plus un vecteur de features par frame mais une *image* (carte de flot optique ou RGB) pour chaque frame. On encode chaque image avec un même réseau de type Vision Transformer (ViT), ce qui donne un vecteur par frame ; on ajoute un encodage positionnel temporel puis un petit Transformer sur la dimension temps pour fusionner l'information. Enfin on moyenne sur le temps et on classe. Ce modèle exploite directement la géométrie spatiale du flot (patches 2D), au prix d'une entrée plus lourde et d'un autre pipeline de données (ipn\_flow : images de flot par séquence).

L'entrée est un tenseur  $\mathbf{X} \in \mathbb{R}^{B \times T \times C \times H \times W}$  (batch, temps, canaux, hauteur, largeur), typiquement  $T = 8$  frames,  $H = W = 224$ ,  $C = 3$  (flot visualisé en RGB ou cartes  $u, v$ ). Le modèle :

1. **Encodeur par frame** : un backbone ViT (ex. vit\_small\_patch16\_224, sans tête, global\_pool="") appliqué à chaque frame ; sortie patch tokens  $\rightarrow$  moyenne sur les patches  $\rightarrow$  vecteur  $\mathbf{e}_t \in \mathbb{R}^d$  par frame ( $d = 384$ ).
2. **Encodage positionnel temporel** : vecteur appris  $\mathbf{E}_{\text{temp}} \in \mathbb{R}^{1 \times T \times d}$  ajouté aux  $\mathbf{e}_t$ .
3. **Transformer temporel** : TransformerEncoder (quelques couches, attention + FFN, GELU, norm-first) sur la séquence  $(\mathbf{e}_1 + E_1, \dots, \mathbf{e}_T + E_T)$  ; sortie  $(B, T, d)$ .
4. **Aggrégation et tête** : LayerNorm sur la moyenne temporelle  $\frac{1}{T} \sum_t \mathbf{o}_t$ , puis Dropout et Linear( $d, K$ )  $\rightarrow$  logits.

Contrairement aux autres modèles de cette section, le Temporal ViT est entraîné avec le dataset **ipn\_flow** (images de flot par séquence) et non le manifest pose+optflow ; les métriques rapportées ne sont donc pas directement comparables.

## 5 Expériences et résultats

### 5.1 Protocole

- **Données** : manifest IPN Hand, split train/val/test ; les métriques rapportées sont sur l'ensemble de **validation** (best epoch selon l'exactitude sur la validation).
- **Entraînement** : 20 epochs, AdamW (lr  $5 \times 10^{-4}$ , weight decay 0.01), CosineAnnealingLR, batch size 32, gradient clipping (max norm 1). Une seule graine utilisée pour les runs rapportés. *Note* : Temporal ViT a été entraîné avec batch size 8 et lr  $10^{-4}$  (dataset ipn\_flow).
- **Métriques** : exactitude (accuracy), précision/rappel/F1 en macro et pondéré (weighted).

### 5.2 Comparaison des modèles

Les tableaux suivants résument les **meilleures métriques** (best epoch) pour chaque modèle sur la validation. Les runs proviennent des dossiers `models/<model>_<date>_<heure>/` (metrics.json). Temporal ViT est entraîné sur ipn\_flow ; ses chiffres ne sont pas directement comparables.

TABLE 1 – Comparaison des performances (validation) — métriques globales.

| Modèle                    | Accuracy | P (macro) | R (macro) | F1 (macro) | F1 (weighted) | Val. loss |
|---------------------------|----------|-----------|-----------|------------|---------------|-----------|
| CNN-LSTM                  | 0.883    | 0.817     | 0.836     | 0.823      | 0.884         | 0.595     |
| Temporal Transformer      | 0.850    | 0.742     | 0.764     | 0.748      | 0.850         | 0.610     |
| Temporal ViT <sup>1</sup> | 0.728    | 0.611     | 0.602     | 0.592      | 0.723         | 1.06      |
| ST-GCN-Opt                | 0.665    | 0.465     | 0.300     | 0.311      | 0.597         | 1.365     |
| ST-GCN                    | 0.538    | 0.246     | 0.226     | 0.196      | 0.458         | 1.549     |

TABLE 2 – Perte d'entraînement et epoch du meilleur modèle (validation).

| Modèle               | Train loss (best) | Best epoch |
|----------------------|-------------------|------------|
| CNN-LSTM             | 0.0629            | 20         |
| Temporal Transformer | 0.3464            | 18         |
| Temporal ViT         | 0.218             | 7          |
| ST-GCN-Opt           | 0.8283            | 17         |
| ST-GCN               | 1.6031            | 20         |

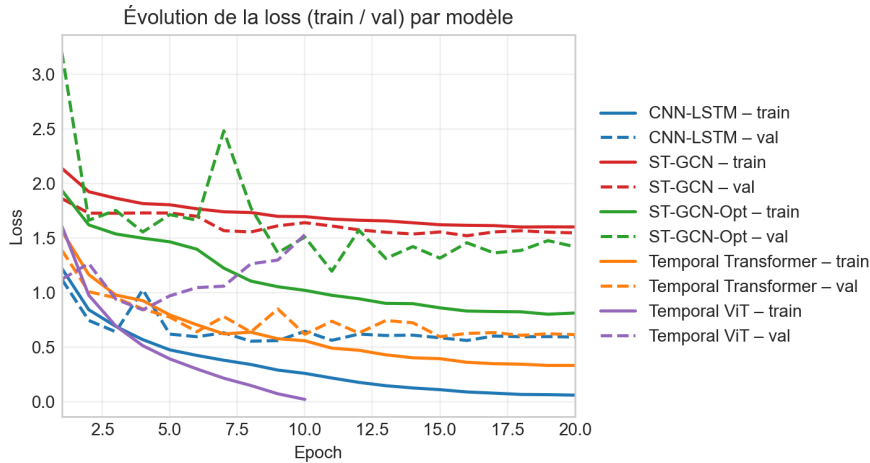


FIGURE 1 – Évolution de la perte d'entraînement et de validation par epoch pour chaque modèle. En trait plein : train loss ; en pointillés : val loss.

### 5.3 Analyse

- **CNN-LSTM** et **Temporal Transformer** obtiennent les meilleures performances (exactitude  $\approx 85\text{--}88\%$ , F1 macro  $\approx 0.75\text{--}0.82$ ), en utilisant directement le vecteur unifié pose+optflow par frame. La représentation séquentielle standard convient bien au jeu IPN Hand avec les features extraites.
- **Pourquoi CNN-LSTM dépasse le Temporal Transformer** (accuracy 88,3% vs 85,0%, F1 macro 0,82 vs 0,75) : (1) la convolution 1D et la récurrence LSTM introduisent un biais d'induction fort pour les motifs temporels *locaux* (phases successives d'un geste), ce qui peut mieux généraliser avec peu de données ; (2) le Transformer repose sur l'attention globale et a généralement besoin de plus de données ou d'une régularisation plus forte pour éviter le surapprentissage ; (3) la perte d'entraînement à la meilleure epoch

1. Temporal ViT : entrée images de flot (ipn\_flow), non directement comparable.

est nettement plus basse pour CNN-LSTM (0,063 vs 0,346), ce qui suggère un meilleur ajustement sans dégradation de la validation. Les deux modèles gèrent les séquences de longueur variable de la même façon (T fixe par batch, masque de padding). En résumé, pour la taille actuelle du jeu IPN Hand et les features utilisées, l’architecture CNN-LSTM offre un meilleur compromis capacité / régularisation que le Temporal Transformer.

- **Temporal ViT** (entrée : images de flot, ipn\_flow) atteint environ 72,8 % d’exactitude et 0,59 de F1 macro dans notre run (best epoch 7). Les performances ne sont pas directement comparables aux modèles pose+optflow car l’entrée et le pipeline de données diffèrent ; le modèle pourrait bénéficier d’un pré-entraînement ou d’un réglage dédié.
- **ST-GCN-Opt** est en retrait (accuracy  $\approx 66.5\%$ , F1 macro  $\approx 0.31$ ), avec une forte variance sur les classes de gestes (bon rappel sur D0X/B0A/B0B, rappel faible sur plusieurs G0x). La fusion multi-branches et le format skeleton/track/optflow pourraient nécessiter un réglage (learning rate, régularisation, ou plus de données) pour converger mieux.
- **ST-GCN** seul atteint seulement  $\approx 53.8\%$  d’exactitude (F1 macro  $\approx 0.20$ ), avec de nombreux rappels à zéro sur les gestes G04–G06, G10, G11. L’utilisation du graphe skeleton+motion sans les branches track/optflow et avec un graphe trivial ( $\mathbf{A} = \mathbf{I}$ ) limite probablement la capacité du modèle sur ce dataset.

En résumé, pour ce pipeline et ce split, les modèles basés sur la séquence de features unifiées (CNN-LSTM, Temporal Transformer) surpassent les modèles à graphe spatio-temporel (ST-GCN, ST-GCN-Opt) dans la configuration actuelle.

## 6 Conclusion

Le projet DOMA fournit un pipeline complet pour la reconnaissance de gestes de la main à partir de vidéos : détection de la main (MediaPipe/YOLO), flot optique (Farneback/RAFT), extraction de métriques (magnitude, direction dominante, concentration) et de pose (track 3D, landmarks), puis classification par plusieurs modèles (CNN-LSTM, Temporal Transformer, ST-GCN, ST-GCN-Opt, Temporal ViT). Les formules de représentation (moyenne circulaire, pooling masqué, blocs ST-GCN) ont été rappelées et les performances sur IPN Hand comparées en tableaux. Les meilleurs résultats sont obtenus avec CNN-LSTM et Temporal Transformer sur les features pose+optflow unifiées ; les architectures à graphe pourraient être améliorées par un graphe spatial plus riche et un réglage dédié.

## Références

1. G. Benitez-Garcia et al., “IPN Hand : A Video Dataset and Benchmark for Real-Time Continuous Hand Gesture Recognition,” in *ICPR*, 2020.
2. G. Farneback, “Two-frame motion estimation based on polynomial expansion,” in *Scandinavian Conference on Image Analysis*, 2003.
3. Y. Yan, S. Xiong, et D. Lin, “Spatial temporal graph convolutional networks for skeleton-based action recognition,” in *AAAI*, 2018.
4. A. Vaswani et al., “Attention is all you need,” in *NeurIPS*, 2017.
5. A. Dosovitskiy et al., “An image is worth 16x16 words : Transformers for image recognition at scale,” in *ICLR*, 2021.